

Task Parallel Matrix Algorithms

1) Matrix Vector Multiplication

2) Matrix Multiplication

Instructor: Satish Puri

Parallel loops

- Matrix vector multiplication

As an example, consider the problem of multiplying a square $n \times n$ matrix $A = (a_{ij})$ by an n -vector $x = (x_j)$. The resulting n -vector $y = (y_i)$ is given by the equation

$$y_i = \sum_{j=1}^n a_{ij} x_j ,$$

P-MAT-VEC(A, x, y, n)

```
1  parallel for  $i = 1$  to  $n$            // parallel loop
2      for  $j = 1$  to  $n$            // serial loop
3           $y_i = y_i + a_{ij} x_j$ 
```

Parallel Recursive Matrix Vector multiplication

P-MAT-VEC(A, x, y, n)

```
1  parallel for  $i = 1$  to  $n$            // parallel loop
2      for  $j = 1$  to  $n$            // serial loop
3           $y_i = y_i + a_{ij}x_j$ 
```

P-MAT-VEC-RECURSIVE(A, x, y, n, i, i')

```
1  if  $i == i'$                        // just one iteration to do?
2      for  $j = 1$  to  $n$                // mimic P-MAT-VEC serial loop
3           $y_i = y_i + a_{ij}x_j$ 
4  else  $mid = \lfloor (i + i')/2 \rfloor$     // parallel divide-and-conquer
5      spawn P-MAT-VEC-RECURSIVE( $A, x, y, n, i, mid$ )
6      P-MAT-VEC-RECURSIVE( $A, x, y, n, mid + 1, i'$ )
7  sync
```

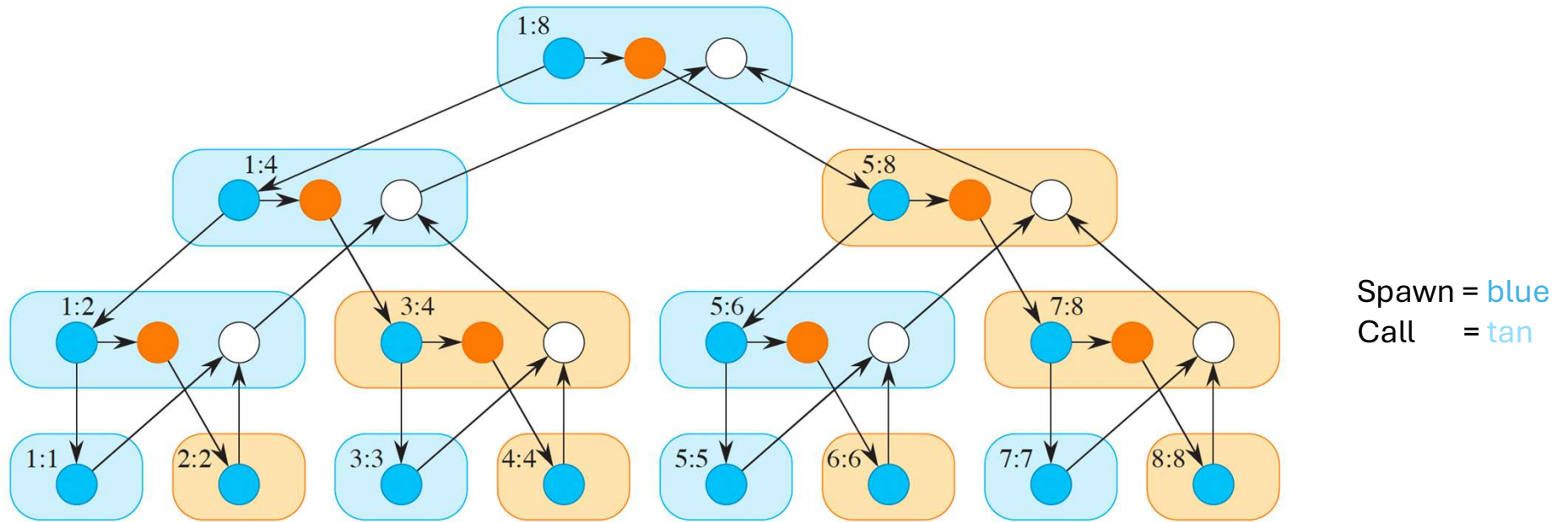


Figure 26.4 A trace for the computation of $\text{P-MAT-VEC-RECURSIVE}(A, x, y, 8, 1, 8)$. The two numbers within each rounded rectangle give the values of the last two parameters (i and i' in the procedure header) in the invocation (spawn, in blue, or call, in tan) of the procedure. The blue circles represent strands corresponding to the part of the procedure up to the spawn of $\text{P-MAT-VEC-RECURSIVE}$ in line 5. The orange circles represent strands corresponding to the part of the procedure that calls $\text{P-MAT-VEC-RECURSIVE}$ in line 6 up to the **sync** in line 7, where it suspends until the spawned subroutine in line 5 returns. The white circles represent strands corresponding to the (negligible) part of the procedure after the **sync** up to the point where it returns.

Analysis Matrix Vector Multiplication

Although recursive spawning doesn't affect the work of a parallel loop asymptotically, we must take it into account when analyzing the span. Consider a parallel loop with n iterations in which the i th iteration has span $iter_{\infty}(i)$. Since the depth of recursion is logarithmic in the number of iterations, the parallel loop's span is

$$T_{\infty}(n) = \Theta(\lg n) + \max \{iter_{\infty}(i) : 1 \leq i \leq n\} .$$

- $\log(n)$ is for dividing the number of rows into halves (outer for loop)

Analysis Matrix Vector Multiplication

For example, let's compute the span of the doubly nested loops in lines 1–3 of P-MAT-VEC. The span for the **parallel for** loop control is $\Theta(\lg n)$. For each iteration of the outer parallel loop, the inner serial **for** loop contains n iterations of line 3. Since each iteration takes constant time, the total span for the inner serial **for** loop is $\Theta(n)$, no matter which iteration of the outer **parallel for** loop it's in. Thus, taking the maximum over all iterations of the outer loop and adding in the $\Theta(\lg n)$ for loop control yields an overall span of $T_\infty n = \Theta(n) + \Theta(\lg n) = \Theta(n)$ for the procedure. Since the work is $\Theta(n^2)$, the parallelism is $\Theta(n^2)/\Theta(n) = \Theta(n)$. (Exercise 26.1-7 asks you to provide an implementation with even more parallelism.)

Race conditions

- Deterministic = same output every time code runs
- Non-deterministic = possibly different output every time code runs with same input

A determinacy race occurs when two logically parallel instructions access the same memory location and at least one of the instructions modifies the value stored in the location.

Why race bugs matter?

Famous race bugs include the Therac-25 radiation therapy machine, which killed three people and injured several others, and the Northeast Blackout of 2003, which left over 50 million people in the United States without power. These pernicious bugs are notoriously hard to find. You can run tests in the lab for days without a failure, only to discover that your software sporadically crashes in the field, sometimes with dire consequences.

Example of Race Condition

```
RACE-EXAMPLE( )
```

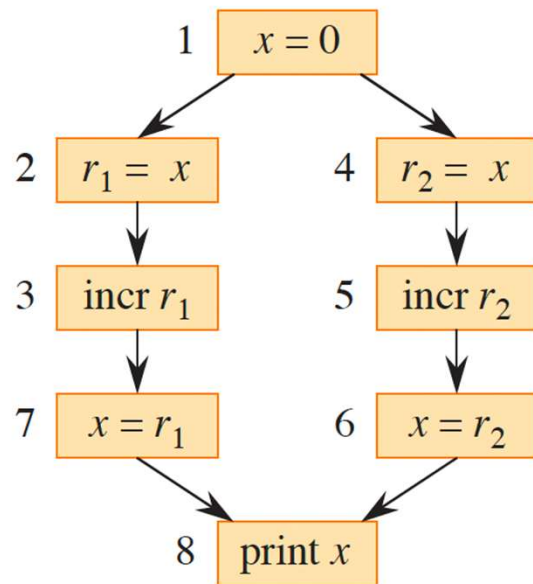
```
1   $x = 0$ 
```

```
2  parallel for  $i = 1$  to 2
```

```
3       $x = x + 1$            // determinacy race
```

```
4  print  $x$ 
```

- Load x from memory into one of the processor's registers.
- Increment the value in the register.
- Store the value in the register back into x in memory.



(a)

step	x	r_1	r_2
1	0	—	—
2	0	0	—
3	0	1	—
4	0	1	0
5	0	1	1
6	1	1	1
7	1	1	1

(b)

RACE-EXAMPLE()

1 $x = 0$

2 **parallel for** $i = 1$ to 2

3 $x = x + 1$

4 **print** x

// determinacy

Figure 26.5 Illustration of the determinacy race in RACE-EXAMPLE. (a) A trace showing the dependencies among individual instructions. The processor registers are r_1 and r_2 . Instructions unrelated to the race, such as the implementation of loop control, are omitted. (b) An execution sequence that elicits the bug, showing the values of x in memory and registers r_1 and r_2 for each step in the execution sequence.

Matrix vector multiplication

P-MAT-VEC-WRONG(A, x, y, n)

1 **parallel for** $i = 1$ **to** n

2 **parallel for** $j = 1$ **to** n

3 $y_i = y_i + a_{ij}x_j$ *// determinacy race*

Improved Matrix Vector Multiplication

Give a parallel algorithm to multiply an $n \times n$ matrix by an n -vector that achieves $\Theta(n^2 / \lg n)$ parallelism while maintaining $\Theta(n^2)$ work.

- Slow version

P-MAT-VEC(A, x, y, n)

1 **parallel for** $i = 1$ **to** n

2 **for** $j = 1$ **to** n

3 $y_i = y_i + a_{ij}x_j$

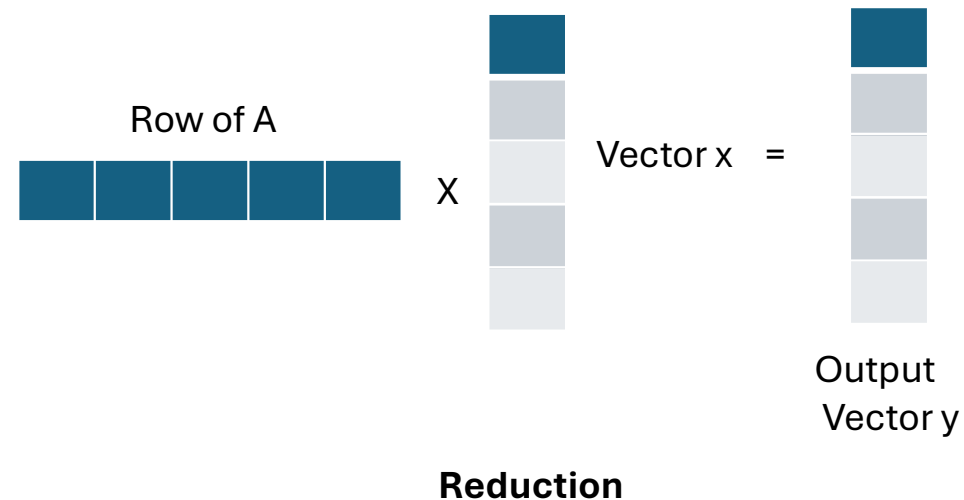
// parallel loop

// serial loop

Matrix **a** vector **x** multiplication

Algorithm 2 MAT-VEC(A, x)

```
1:  $n = A.rows$ 
2: let  $y$  be a new vector of length  $n$ 
3: parallel for  $i = 1$  to  $n$  do
4:    $y_i = 0$ 
5: end
6: parallel for  $i = 1$  to  $n$  do
7:    $y_i = \text{P-PROD}(a_i, x, 1, n)$ 
8: end
9: return  $y$ 
```



Matrix **a** vector **x** multiplication

Algorithm 2 MAT-VEC(A, x)

```
1:  $n = A.rows$ 
2: let  $y$  be a new vector of length  $n$ 
3: parallel for  $i = 1$  to  $n$  do
4:    $y_i = 0$ 
5: end
6: parallel for  $i = 1$  to  $n$  do
7:    $y_i = \text{P-PROD}(a_i, x, 1, n)$ 
8: end
9: return  $y$ 
```

Algorithm 1 P-PROD(a, x, j, j')

```
1: if  $j == j'$  then
2:   return  $a[j] \cdot x[j]$ 
3: end if
4:  $mid = \left\lfloor \frac{j+j'}{2} \right\rfloor$ 
5:  $a' = \text{spawn P-PROD}(a, x, j, mid)$ 
6:  $x' = \text{P-PROD}(a, x, mid+1, j')$ 
7: sync
8: return  $a' + x'$ 
```

Since the work of each task is $o(\lg(n))$, that doesn't affect the T_∞ runtime. The parallelism is equal to the work over the span, so it is $\Theta(n^2/\lg(n))$.

Multiply a row of Matrix **a** with vector **x** via Reduction

We'll parallelize the for loop of lines 6-7 in a way which won't incur races. With the algorithm *P-PROD* given below, it will be easy to rewrite the code. For notation, let a_i denote the i^{th} row of the matrix A .

Algorithm 1 P-PROD(a, x, j, j')

```
1: if  $j == j'$  then  
2:   return  $a[j] \cdot x[j]$   
3: end if  
4:  $mid = \left\lfloor \frac{j+j'}{2} \right\rfloor$   
5:  $a' = \text{spawn P-PROD}(a, x, j, mid)$   
6:  $x' = \text{P-PROD}(a, x, mid+1, j')$   
7: sync  
8: return  $a' + x'$ 
```

Sequential Matrix Matrix Multiply

MATRIX-MULTIPLY(A, B, C, n)

```
1  for  $i = 1$  to  $n$                                 // compute entries in each of  $n$  rows
2      for  $j = 1$  to  $n$                                 // compute  $n$  entries in row  $i$ 
3          for  $k = 1$  to  $n$ 
4               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$  // add in another term of equation (4.1)
```

Parallel Matrix Multiplication Algorithms

- Three parallel algorithms
- Different Asymptotic Run-times

Parallel Matrix Multiplication (Simple version)

P-MATRIX-MULTIPLY(A, B, C, n)

```
1  parallel for  $i = 1$  to  $n$            // compute entries in each of  $n$  rows
2      parallel for  $j = 1$  to  $n$        // compute  $n$  entries in row  $i$ 
3          for  $k = 1$  to  $n$ 
4               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$  // add in another term of equation (4.1)
```

Parallelism in Matrix Multiplication

$$T_1(n) = \Theta(n^3).$$

$$\Theta(\lg n) + \Theta(\lg n) + \Theta(n) = \Theta(n).$$

$$T_\infty(n) = \Theta(n).$$

Parallelism $\Theta(n^3)/\Theta(n) = \Theta(n^2)$.

```
P-MATRIX-MULTIPLY(A, B, C, n)
1  parallel for i = 1 to n
2      parallel for j = 1 to n
3          for k = 1 to n
4              cij = cij + aik · bkj
```

Block Matrix Multiplication

- Sequential
- Parallel

The divide step views each of the $n \times n$ matrices A , B , and C as four $n/2 \times n/2$ submatrices:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}. \quad (4.2)$$

Then we can write the matrix product as

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad (4.3)$$

$$= \begin{pmatrix} A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ A_{21} \cdot B_{11} + A_{22} \cdot B_{21} & A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{pmatrix}, \quad (4.4)$$

which corresponds to the equations

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}, \quad (4.5)$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}, \quad (4.6)$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}, \quad (4.7)$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}. \quad (4.8)$$

Equations (4.5)–(4.8) involve eight $n/2 \times n/2$ multiplications and four additions of $n/2 \times n/2$ submatrices.

MATRIX-MULTIPLY-RECURSIVE(A, B, C, n)

```
1  if  $n == 1$ 
2    // Base case.
3       $c_{11} = c_{11} + a_{11} \cdot b_{11}$ 
4      return
5    // Divide.
6    partition  $A, B$ , and  $C$  into  $n/2 \times n/2$  submatrices
       $A_{11}, A_{12}, A_{21}, A_{22}; B_{11}, B_{12}, B_{21}, B_{22};$ 
      and  $C_{11}, C_{12}, C_{21}, C_{22};$  respectively
7    // Conquer.
8    MATRIX-MULTIPLY-RECURSIVE( $A_{11}, B_{11}, C_{11}, n/2$ )
9    MATRIX-MULTIPLY-RECURSIVE( $A_{11}, B_{12}, C_{12}, n/2$ )
10   MATRIX-MULTIPLY-RECURSIVE( $A_{21}, B_{11}, C_{21}, n/2$ )
11   MATRIX-MULTIPLY-RECURSIVE( $A_{21}, B_{12}, C_{22}, n/2$ )
12   MATRIX-MULTIPLY-RECURSIVE( $A_{12}, B_{21}, C_{11}, n/2$ )
13   MATRIX-MULTIPLY-RECURSIVE( $A_{12}, B_{22}, C_{12}, n/2$ )
14   MATRIX-MULTIPLY-RECURSIVE( $A_{22}, B_{21}, C_{21}, n/2$ )
15   MATRIX-MULTIPLY-RECURSIVE( $A_{22}, B_{22}, C_{22}, n/2$ )
```

MATRIX-MULTIPLY-RECURSIVE(A, B, C, n)

```
1  if  $n == 1$ 
2    // Base case.
3       $c_{11} = c_{11} + a_{11} \cdot b_{11}$ 
4      return
5    // Divide.
6    partition  $A, B$ , and  $C$  into  $n/2 \times n/2$  submatrices
       $A_{11}, A_{12}, A_{21}, A_{22}; B_{11}, B_{12}, B_{21}, B_{22};$ 
      and  $C_{11}, C_{12}, C_{21}, C_{22};$  respectively
7    // Conquer.
8    MATRIX-MULTIPLY-RECURSIVE( $A_{11}, B_{11}, C_{11}, n/2$ )
9    MATRIX-MULTIPLY-RECURSIVE( $A_{11}, B_{12}, C_{12}, n/2$ )
10   MATRIX-MULTIPLY-RECURSIVE( $A_{21}, B_{11}, C_{21}, n/2$ )
11   MATRIX-MULTIPLY-RECURSIVE( $A_{21}, B_{12}, C_{22}, n/2$ )
12   MATRIX-MULTIPLY-RECURSIVE( $A_{12}, B_{21}, C_{11}, n/2$ )
13   MATRIX-MULTIPLY-RECURSIVE( $A_{12}, B_{22}, C_{12}, n/2$ )
14   MATRIX-MULTIPLY-RECURSIVE( $A_{22}, B_{21}, C_{21}, n/2$ )
15   MATRIX-MULTIPLY-RECURSIVE( $A_{22}, B_{22}, C_{22}, n/2$ )
```

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21} ,$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22} ,$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21} ,$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22} .$$

Master's theorem

Draw a recurrence tree

Recurrence Relation

$$T(n) = a T(n/b) + f(n)$$

Height of the recurrence tree = $\log_b n$

Number of children = $a^{\log_b n} = n^{\log_b a}$

a^0

a^1

a^2

a^3

..

$$a^{\text{height}} = a^{\log_b n} = n^{\log_b a}$$

Master's theorem

Recurrence Relation

$$T(n) = a T(n/b) + f(n)$$

Compare two values

- Number of children in a recursive tree

$$n^{\log_b a}$$

- $f(n)$ is Merge cost

Case 1 $n^{\log_b a} > f(n)$	Case 2 $n^{\log_b a} = f(n)$	Case 3 $n^{\log_b a} < f(n)$
$T(n) = O(n^{\log_b a})$	$T(n) = O(n^{\log_b a} * \log n)$	$T(n) = O(f(n))$

Master Theorem

Theorem 4.1 (Master theorem)

Let $a > 0$ and $b > 1$ be constants, and let $f(n)$ be a driving function that is defined and nonnegative on all sufficiently large reals. Define the recurrence $T(n)$ on $n \in \mathbb{N}$ by

$$T(n) = aT(n/b) + f(n) , \tag{4.17}$$

If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.

Parallel Matrix Multiplication Algorithm 2

- Recursive block matrix multiplication

P-MATRIX-MULTIPLY-RECURSIVE(A, B, C, n)

```
1  if  $n == 1$                                 // just one element in each matrix?
2       $c_{11} = c_{11} + a_{11} \cdot b_{11}$ 
3      return
4  let  $D$  be a new  $n \times n$  matrix    // temporary matrix
5  parallel for  $i = 1$  to  $n$         // set  $D = 0$ 
6      parallel for  $j = 1$  to  $n$ 
7           $d_{ij} = 0$ 
8  partition  $A, B, C$ , and  $D$  into  $n/2 \times n/2$  submatrices
     $A_{11}, A_{12}, A_{21}, A_{22}; B_{11}, B_{12}, B_{21}, B_{22}; C_{11}, C_{12}, C_{21}, C_{22};$ 
    and  $D_{11}, D_{12}, D_{21}, D_{22}$ ; respectively
9  spawn P-MATRIX-MULTIPLY-RECURSIVE( $A_{11}, B_{11}, C_{11}, n/2$ )
10 spawn P-MATRIX-MULTIPLY-RECURSIVE( $A_{11}, B_{12}, C_{12}, n/2$ )
11 spawn P-MATRIX-MULTIPLY-RECURSIVE( $A_{21}, B_{11}, C_{21}, n/2$ )
12 spawn P-MATRIX-MULTIPLY-RECURSIVE( $A_{21}, B_{12}, C_{22}, n/2$ )
13 spawn P-MATRIX-MULTIPLY-RECURSIVE( $A_{12}, B_{21}, D_{11}, n/2$ )
14 spawn P-MATRIX-MULTIPLY-RECURSIVE( $A_{12}, B_{22}, D_{12}, n/2$ )
15 spawn P-MATRIX-MULTIPLY-RECURSIVE( $A_{22}, B_{21}, D_{21}, n/2$ )
16 spawn P-MATRIX-MULTIPLY-RECURSIVE( $A_{22}, B_{22}, D_{22}, n/2$ )
17 sync                                // wait for spawned submatrix products
18 parallel for  $i = 1$  to  $n$             // update  $C = C + D$ 
19     parallel for  $j = 1$  to  $n$ 
20          $c_{ij} = c_{ij} + d_{ij}$ 
```

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21} ,$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22} ,$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21} ,$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22} .$$

Work done by Recursive Matrix Multiplication

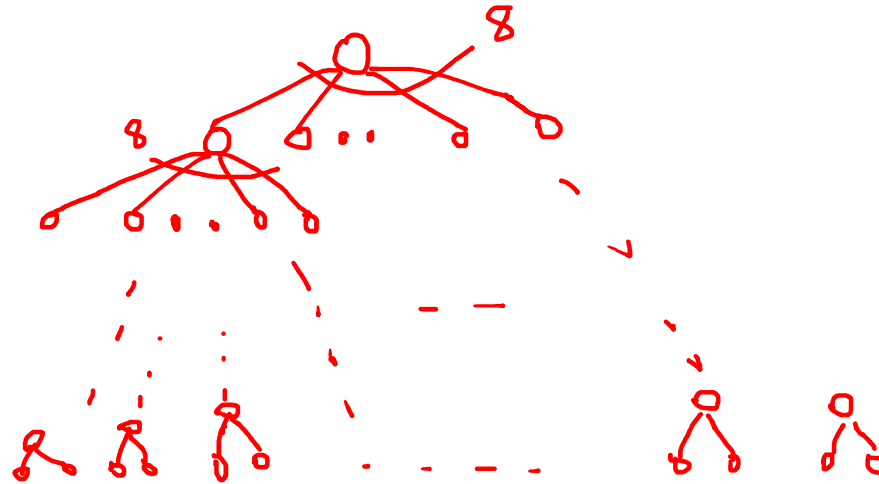
temporary matrix D in $\Theta(n^2)$ time, partitions in $\Theta(1)$ time, performs eight recursive multiplications of $n/2 \times n/2$ matrices, and finishes up with the $\Theta(n^2)$ work from adding two $n \times n$ matrices. Thus the work outside the spawned recursive calls is $\Theta(n^2)$, and the recurrence for the work $M_1(n)$ becomes

$$\begin{aligned} M_1(n) &= 8M_1(n/2) + \Theta(n^2) \\ &= \Theta(n^3) \end{aligned}$$

Using Master Theorem, case 1.

Analyzing parallel recursive MatMul algorithm

- Work
- Span



$$M_{\infty}(n) = M_{\infty}(n/2) + \Theta(\lg n) . \quad (26.6)$$

Since this recurrence falls under case 2 of the master theorem with $k = 1$, the solution is $M_{\infty}(n) = \Theta(\lg^2 n)$.

Theorem 4.1 (Master theorem)

Let $a > 0$ and $b > 1$ be constants, and let $f(n)$ be a driving function that is defined and nonnegative on all sufficiently large reals. Define the recurrence $T(n)$ on $n \in \mathbb{N}$ by

$$T(n) = aT(n/b) + f(n) , \quad (4.17)$$

2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$, then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.

Parallelism of Mat Mul

$$M_{\infty}(n) = M_{\infty}(n/2) + \Theta(\lg n) . \quad (26.6)$$

Since this recurrence falls under case 2 of the master theorem with $k = 1$, the solution is $M_{\infty}(n) = \Theta(\lg^2 n)$.

The parallelism of P-MATRIX-MULTIPLY-RECURSIVE is $M_1(n)/M_{\infty}(n) = \Theta(n^3 / \lg^2 n)$, which is huge. (Problem 26-2 asks you to simplify this parallel algorithm at the expense of just a little less parallelism.)

Can we do better time complexity in MatMul?

- 1st algorithm – $O(N^2)$ Parallelism
- 2nd algorithm - $O(N^3 / \log^2 N)$ Parallelism using “blocking” matrix idea
- New: 3rd algorithm
 $O(N^3 / \log N)$ Parallelism

Can we do better time complexity in MatMul?

P-MATRIX-MULTIPLY(A, B, C, n)

```
1  parallel for  $i = 1$  to  $n$            // compute entries in each of  $n$  rows
2      parallel for  $j = 1$  to  $n$        // compute  $n$  entries in row  $i$ 
3          for  $k = 1$  to  $n$ 
4               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$  // add in another term of equation (4.1)
```

Can we do better time complexity in MatMul?

Algorithm 4 MODIFIED-P-SQUARE-MATRIX-MULTIPLY

$n = A.\text{rows}$

let C be a new $n \times n$ matrix

parallel for $i=1$ to n **do**

parallel for $j=1$ to n **do**

$c_{i,j} = \text{P-DOT-PROD}(A_{i,\cdot}, B_{\cdot,j}, 1, n)$

end

end

return C

Parallel dot product (matrix row with mat col)

Algorithm 3 P-DOT-PROD($v, w, \text{low}, \text{high}$)

if $\text{low} == \text{high}$ **then**

return $v[\text{low}] \times w[\text{low}]$

end if

$\text{mid} = \left\lfloor \frac{\text{low} + \text{high}}{2} \right\rfloor$

$x = \text{spawn P-DOT-PROD}(v, w, \text{low}, \text{mid})$

$y = \text{P-DOT-PROD}(v, w, \text{mid} + 1, \text{high})$

sync

return $x + y$

Algorithm 4 MODIFIED-P-SQUARE-MAT

```
n = A.rows
let C be a new  $n \times n$  matrix
parallel for i=1 to n do
    parallel for j=1 to n do
         $c_{i,j} = \text{P-DOT-PROD}(A_{i,.}, B_{.,j}, 1, n)$ 
    end
end
return C
```

Algorithm 3 P-DOT-PROD(v,w,low,high)

```
if low == high then
    return v[low]  $\times$  w[low]
end if
mid =  $\left\lfloor \frac{low+high}{2} \right\rfloor$ 
x = spawn P-DOT-PROD(v,w,low,mid)
y = P-DOT-PROD(v,w,mid+1,high)
sync
return x+y
```

Parallel dot product (matrix row with mat col)

Algorithm 3 P-DOT-PROD($v, w, low, high$)

```
if low == high then
    return  $v[low] \times w[low]$ 
end if
mid =  $\left\lfloor \frac{low+high}{2} \right\rfloor$ 
x = spawn P-DOT-PROD( $v, w, low, mid$ )
y = P-DOT-PROD( $v, w, mid+1, high$ )
sync
return  $x+y$ 
```

Span = $O(\log N)$

MatMul Parallelism

- $O(N^3/\log N)$